

Turing Machines

By: Gustavo F., Logan N., Thomas N.,
Ethan K., & Christopher M.

1. Formal Definition of a Turing Machine

Formal Definition of a Turing Machine

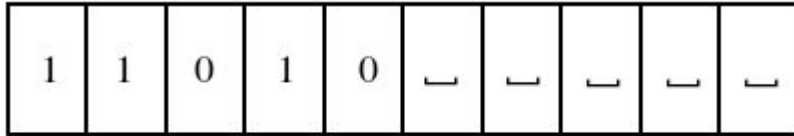
A Turing machine is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and:

1. Q is the set of states
2. Σ is the input alphabet not containing the blank symbol $\sqcup$
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$
4. $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function
5. $q_0 \in Q$ is the start state
6. $q_{\text{accept}} \in Q$ is the accept state
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$

Role of Tape

The tape is the machine's memory:

- It is divided into infinitely many cells, giving it unlimited memory.
- Each cell holds one symbol from the tape alphabet Γ
- It stores the input string, followed by blank symbols



Role of Head Movement

The head movement reads, writes, and moves across the tape:

1. Read - Looks at the symbol currently on the tape cell
2. Write - May replace the symbol in that cell.
3. Move - Moves one cell to the left or right
 - L $\rightarrow$ go left
 - R $\rightarrow$ go right

2. Basic Operations of Turing Machines

How a TM reads, writes, and moves

Reading: The machine is able to read the tape by using the machine's head

Writing: Overwrites the current cell under the head

Moves: After writing, the head shifts left or right exactly one cell based on the transition table

Transition:

$$\delta(q, a) = (q', b, D)$$

- Current state q
- Read symbol a
- Write symbol b
- Move head direction $D = L$ or R
- Move to next state q'

Read a 1

$$\delta(q_{\text{current}}, 1) = (q_{\text{next}}, X, R)$$

	<u>1</u>	1	0	0	
--	----------	---	---	---	--

Write an X

	<u>X</u>	1	0	0	
--	----------	---	---	---	--

After writing, move head right; Now on to next state

	X	<u>1</u>	0	0	
--	---	----------	---	---	--

Configuration

- Represents the current state of the machine as $u q v$
- q represents the current state
- uv represents the two strings:

	x	<u>1</u>	0	0	
--	---	----------	---	---	--

$X q_2 100$ yields $XX q_3 00$

	x	x	<u>0</u>	0	
--	---	---	----------	---	--

3. Simple Example of a Turing Machine

Turing Machine TM recognize Language

$$B = \{a^n b^n c^n \mid n \geq 0\}$$

How we process the string:

1. Scan right crossing off single a,b,c replacing with symbol (a, b, c)
 - a. If each symbol is removed on a pass and no original symbols remain, accept.
 - b. If some symbols are removed on pass and non remain, reject
 - c. If all symbols remain we move the head back until we hit an [a] and repeat the process.

<u>a</u>	a	a	<u>b</u>	b	b	<u>c</u>	c	c	_	...
----------	---	---	----------	---	---	----------	---	---	---	-----

Repeat

<u>a</u>	<u>a</u>	a	<u>b</u>	<u>b</u>	b	<u>c</u>	<u>c</u>	c	_	...
----------	----------	---	----------	----------	---	----------	----------	---	---	-----

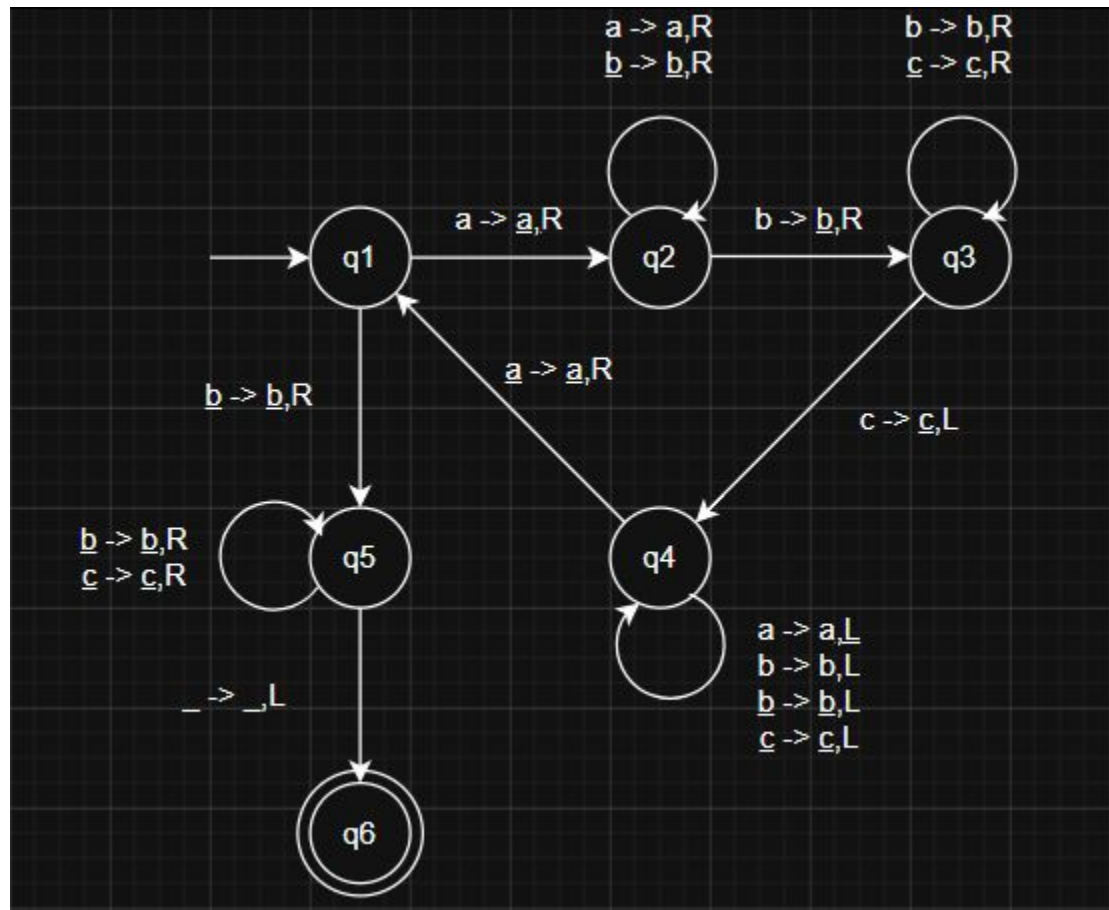
Repeat

<u>a</u>	<u>a</u>	<u>a</u>	<u>b</u>	<u>b</u>	<u>b</u>	<u>c</u>	<u>c</u>	c	_	...
----------	----------	----------	----------	----------	----------	----------	----------	---	---	-----

Repeat

<u>a</u>	<u>a</u>	<u>a</u>	<u>b</u>	<u>b</u>	<u>b</u>	<u>c</u>	<u>c</u>	<u>c</u>	_	...
----------	----------	----------	----------	----------	----------	----------	----------	----------	---	-----

ACCEPT



a	a	a	b	b	b	c	c	c	□	...
---	---	---	---	---	---	---	---	---	---	-----

4. Variations of Turing Machines

Multitape Turing Machines

A multitape Turing machine is a regular Turing machine with multiple tapes and:

- Each tape having its own read and write head
- Input starting on tape 1
- Rest of the tapes starting out blank
- Head starts at first cell for all tapes

For a multitape turing machine $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where k is the amount of tapes, every component is identical to a regular Turing machine except the transition function:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, L, R, \dots, L)$$

Multi-track Turing Machines

A multi-track Turing machine has k amount of tapes and:

- There is only one read and write head
- The head always looks at the k^{th} cell for all tapes (if the head is on the 3rd cell on the first tape, it will be on the 3rd cell for all k tapes)
- All k tapes will move the same direction for each transition

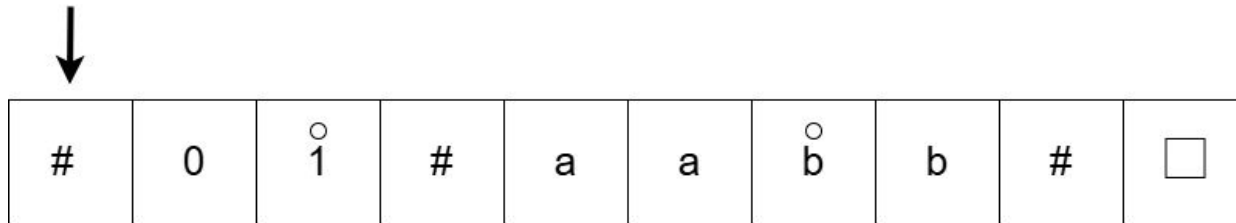
For a multi-track turing machine $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where k is the amount of tapes, each component is identical to a regular Turing machine except the transition function:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}$$

Multitape Turing Machines Equivalence

Multitape Turing machines are equivalent to regular turing machines:

1. A multitape turing machine that only uses one tape is just like a regular Turing machine
2. To simulate a multitape Turing machine, a regular Turing machine will:
 - a. Separate input of each tape on the single tape using a delimiter (e.g. “#”)
 - b. To keep track of the heads of each head from the multitape TM, “dotted” symbols will be new symbols added to the tape alphabet to represent them
 - c. The regular TM head moves to each “dotted” symbol and does an applicable transition for each symbol. Each set of regular TM transitions derive from a transition from the multitape TM.
 - d. If the head moves to the right on a delimiter, everything from the right of it to the last delimiter will be shifted one cell to the right and a blank symbol will be written to where the head landed on



Multi-track Turing Machines Equivalence

Multi-track Turing machines are equivalent to regular Turing machines:

1. A multi-track Turing machine that only uses one tape is just like a regular Turing machine
2. For a regular Turing machine to simulate a multi-track Turing machine, it will follow the same process used for a regular Turing machine to simulate a multitape Turing machine. The only difference is the head will move the same direction from each “dotted” symbol for each set of transitions. A set of transitions is derived from a multi-track Turing machine transition

Multi-head Turing Machines

A multi-head Turing machine has the same components as a multitape Turing machine except:

- There is only one tape
- There are k amount of read/write heads
- Each head moves independently from one another
- If multiple heads write to the same cell, the model must define some kind of priority

Multi-head Turing Machines Equivalence

Multi-head Turing machines are equivalent to regular turing machines:

1. A multi-head Turing machine that just uses one head is just like a regular Turing machine
2. To simulate a multi-head Turing machine, a regular Turing machine will:
 - a. Keep track of each head from the multi-head TM where “dotted” symbols will be new symbols added to the tape alphabet to represent them
 - b. The head moves to each “dotted” symbol and does an applicable transition for each symbol. Each set of transitions depends on a transition from the multi-head TM. A multi-head TM transition will cause k amount of transitions for the regular TM where k is the amount of heads from the multi-head TM

Non-deterministic Turing Machines

Non-deterministic Turing machines recognize languages non-deterministically, meaning:

- The computation is a tree whose branches correspond to different possibilities for the machine
- Machine accepts input if a branch of the computation ends in an accept state
- It is a decider if all branches halt

For a non-deterministic Turing machine $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, every component is identical to a regular Turing machine except the transition function:

$$\delta: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$$

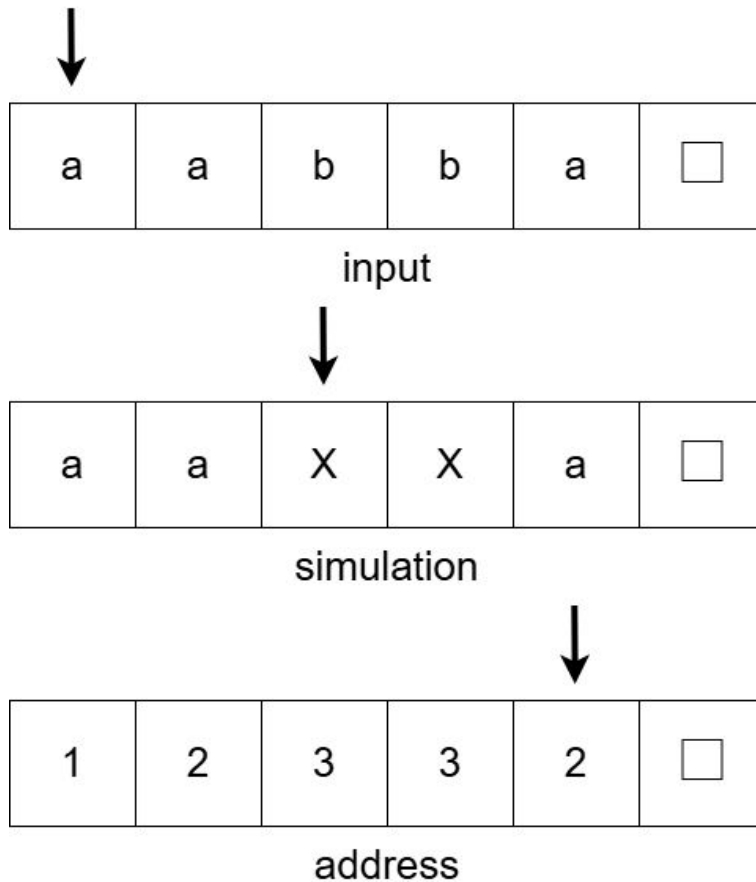
Non-deterministic Turing Machines Equivalence

Non-deterministic Turing machines are equivalent to regular turing machines:

1. A single tape Turing machine is automatically a non-deterministic Turing machine
2. For a non-deterministic Turing machine to be equivalent to a regular Turing machine, we first must create a multitape Turing machine where:
 - a. Tape 1 contains the input w , with tapes 2 and 3 being empty
 - b. Copy contents of tape 1 to tape 2 and initialize string on tape 3 as ϵ
 - c. Use tape 2 to simulate the non-deterministic Turing machine on input w . Before each step of the non-deterministic Turing machine, the head sees the next address symbol on tape 3 to verify the next transition to take. If no address symbols remain on tape 3, the address symbol is invalid, or if it reaches a rejecting configuration, move to step d. If an accepting configuration is encountered, accept.
 - d. Replace string on tape 3 with a new order of address symbols. Simulate the next branch by going back to step b.

Non-deterministic Turing Machines Equivalence

If the multitape TM ends in an accept state, then since a multitape TM is equivalent to a regular TM, a non-deterministic TM is equivalent to a regular TM.



5. Application and Significance of Turing Machines

Church-Turing Thesis Overview

- One of the most important ideas in computer science and in mathematics.
- At its core, its a claim about what it means to “compute” something.
- Two mathematicians arrived at this idea: Alonzo Church and Alan Turing
- Basic Idea: “Any problem that can be solved by a systematic, step-by-step procedure (algorithm) can be solved by a Turing machine and vice versa.”
- In plain english: if you can describe exactly how to solve a problem, then a computer can solve it too.

Decidability and Computability

- Foundational concepts in theoretical computer science, establishing the fundamental limits of what computers can and cannot do.
- Computability: A problem is computable if there exists a Turing Machine that can produce the correct answer for every valid input and then eventually halt. If no Turing Machine exists then the problem is uncomputable.
- Decidability: A problem is decidable if a Turing Machine always halts on every input and returns either “yes” or “no” correctly. If no Turing Machine exists then the problem is undecidable.

Real World Relevance

- These concepts have real world relevance.
- Software verification - you cannot build a tool that checks whether a piece of code has bugs or infinite loops
- Security analysis - many malware detection problems reduce to undecidable problems.
- Artificial Intelligence & Machine Learning - You cannot formally verify the behavior of an arbitrary neural network on all possible inputs. This is because of computability limits.